Reimagining p-Adic Architecture to Accommodate 1-Bit LAMB

Distributed Optimization Bottlenecks and the Mechanics of 1-Bit LAMB

Distributed optimization of large-scale deep neural networks is fundamentally constrained by network communication overhead[1]. As foundation models expand to hundreds of billions of parameters, distributed data-parallel training rapidly transitions from a compute-bound workload to a network bandwidth-bound workload[2]. The Layer-wise Adaptive Moments optimizer for Batch training (LAMB) was formulated to stabilize large-batch execution by dynamically adapting learning rates across distinct layers based on the ratio between the Euclidean norm of the model weights and the norm of the adaptive update vector[2]. Despite enabling effective batch scaling across thousands of nodes, standard LAMB incurs severe latency penalties because it mandates the exchange of uncompressed 32-bit or 16-bit floating-point gradient tensors across cluster interconnects during collective AllReduce operations[2].

Standard gradient compression strategies, notably 1-bit Stochastic Gradient Descent (SGD) with local error feedback, compress communication payloads by transmitting only the 1-bit sign of each coordinate alongside a scalar scaling factor[5]. When applied to adaptive optimizers such as Adam or LAMB, direct error-compensated 1-bit compression fails catastrophically[2]. This divergence is rooted in the non-linear dependency of the second-moment coordinate scaling term on the stochastic gradient[5]. In adaptive optimizers, the uncompressed variance update follows:

$$v_{t+1} = \beta_2 v_t + (1 - \beta_2) g_t^2$$

When an error-compensated gradient $C[g_t + \delta_{t-1}] = g_t + \delta_{t-1} - \delta_t$ is substituted into this non-linear operator, the squared error term expands into:

$$(g_t + \delta_{t-1} - \delta_t)^2 = g_t^2 + (\delta_{t-1} - \delta_t)^2 + 2g_t(\delta_{t-1} -$$

Unlike the linear update of SGD, where consecutive error terms $\delta_{t-1} - \delta_t$ cancel out in a telescoping sum over optimization steps, the quadratic error term $(\delta_{t-1} - \delta_t)^2$ and the cross-term cannot cancel[5]. As a consequence, historical compression noise accumulates

monotonically within the second-moment accumulator, skewing coordinate-wise preconditioners and destroying convergence[5].
To resolve this limitation, 1-bit LAMB implements a two-stage training paradigm combined with adaptive layerwise thresholding[2]. During the initial warmup phase, the optimizer executes full-precision LAMB, allowing both the first moment and second moment to stabilize along smooth trajectories[2]. Once the second moment stabilizes at step T_w, the variance tensor is frozen:

$$v^{(l)} = v^{(l)}_{T_w}$$

Freezing this tensor linearizes the subsequent gradient dependency, re-establishing the telescoping property required for local error-feedback compensation[2]. During the subsequent compression phase, the first-moment momentum vector m_t is compressed into single-bit representations using local error compensation[2].

Operational Phase	Algorithmic Mechanism	Communication Volume	Numerical Dynamics
Warmup Stage ($t < T_w$)	Uncompressed LAMB with dense AllReduce[2]	Full precision (FP32/FP16 tensors)[2]	Rapid convergence of second moment toward steady state[5]
State Transition ($t = T_w$)	Crystallize second moment: $v^{(l)} =$ [cite: 2]	Single synchronization barrier[2]	Freezes variance metric, linearizing update dependencies[5]
Compression Stage ($t \geq T_w$)	1-bit momentum compression with error feedback[2]	1 bit per coordinate plus layer scalar norms[2]	Telescoping error cancellation restored via static preconditioning[2]

In this compressed regime, the preconditioned update vector for layer l is computed via:

$$u_t^{(l)} = \frac{C_{\text{avg}}(m_t^{(l)} + e_{t-1}^{(l)})}{\sqrt{v_{T_w}^{(l)}} + \epsilon} + \lambda x_{t-1}^{(l)}$$

Because aggressive 1-bit quantization can trigger severe perturbations in the layer adaptation ratio $r_t^{(l)} = \phi(\|x_t^{(l)}\|_2)/\|u_t^{(l)}\|_2$, the algorithm clamps the scaling ratio within a bounded envelope relative to a moving average $c_{\text{avg}}^{(l)}$ [2]:

$$r_t^{(l)} = \text{clip}\left(r_t^{(l)}, (1 - r_{\text{threshold}})c_{\text{avg}}^{(l)}, (1 + r_{\text{threshold}})c_{\text{avg}}^{(l)}\right)$$

While 1-bit LAMB reduces network communication volume by up to $4.6\times$ and achieves up to $2.8\times$ end-to-end speedups on commodity interconnects, executing this formulation on conventional IEEE-754 floating-point hardware creates new architectural bottlenecks[2]. Processors must repeatedly pack and unpack 1-bit representations, execute high-latency floating-point square roots and divisions for layer scaling, and store wide floating-point error buffers in external memory, introducing severe memory-bandwidth and arithmetic penalties[2].

Foundations of p-Adic Computing and Ultrametric Topologies

Non-Archimedean mathematics offers an alternative computational foundation through the field of p-adic numbers $\mathbb{Q}_p$ [10]. For any prime number p, any non-zero rational number $x \in \mathbb{Q}$ can be uniquely factored as:

$$x = p^v \frac{a}{b}$$

where integers a and b are coprime to p, and $v = v_p(x) \in \mathbb{Z}$ represents the p-adic valuation of x [10]. The p-adic absolute value is defined as:

$$|x|_p = p^{-v_p(x)}, \quad |0|_p = 0$$

Unlike the Archimedean Euclidean metric, the p-adic norm satisfies the strong ultrametric triangle inequality[10]:

$$|x+y|_p \leq \max\left(|x|_p,\ |y|_p\right)$$

with strict equality holding whenever $|x|_p \neq |y|_p$ [10]. The ring of p-adic integers $\mathbb{Z}_p = \{x \in \mathbb{Q}_p : |x|_p \leq 1\}$ constitutes the closed unit ball of the space[10]. In digital hardware, standard two's complement binary integer arithmetic over k-bit words is isomorphic to the quotient ring $\mathbb{Z}_2/2^k\mathbb{Z}_2$ [10]. Consequently, truncated 2-adic arithmetic and standard fixed-point hardware share an identical algebraic foundation[14]. Additions and multiplications propagate carry bits strictly from the least significant bit (LSB) toward the most significant bit (MSB), meaning information flows in a strictly unidirectional, non-Archimedean sequence that eliminates the need for mantissa pre-alignment[10].
Rational numbers can be represented exactly without floating-point rounding errors using Hensel codes $H(p,r)$ [18]. A rational operand c/d, where denominator d is coprime to p, maps onto a finite r-digit p-adic segment via $h = c \cdot d^{-1} \pmod{p^r}$ [18].

Architectural Dimension	Archimedean IEEE-754 Floating-Point	Non-Archimedean Hensel / 2-Adic Fixed-Point
Distance Metric	Euclidean metric: $\|x+y\| \leq \|x\| +$ [cite: 13]	Ultrametric: $\|x+y\|_p \leq$ [cite: 10, 13]
Topological Space	Continuous real line $\mathbb{R}$ [cite: 13, 21]	Rooted hierarchical tree (dendrogram)[10]
Carry Mechanics	Bidirectional; normalizes toward MSB, rounds at	Unidirectional; carries propagate strictly LSB to

	LSB[16]	MSB[10]
Inversion & Division	High-latency digit recurrence (SRT) or iterative FP division	Exact modular inverse via Newton-Hensel lifting[24]
Error Propagation	Diffusive roundoff drift and catastrophic cancellation[5]	Bounded within valuation ideals $p^k\mathbb{Z}_p$ [cite: 10, 13]

Recent theoretical developments demonstrate that continuous gradient optimization can be natively formulated over p-adic parameters by embedding $\mathbb{Q}_p$ into its tight span Γ_p within the Berkovich affine line $\mathbb{A}^1_{\text{an}}$ [13]. Because $\mathbb{Q}_p$ is totally disconnected, classical directional derivatives along continuous Euclidean paths cannot exist[13]. However, its tight span Γ_p forms a path-connected, uniquely geodesic metric tree where internal vertices represent p-adic disks and edges represent scale variations[13]. Continuous gradient descent, momentum, and adaptive updates can descend this tree structure by evaluating directional slopes along tree branches[13].

Compatibility of 2-Adic Fields and 1-Bit Quantized Gradients

The mathematical formulation of 1-bit gradient quantization exhibits intrinsic compatibility with the 2-adic unit sphere $\mathbb{Z}_2^\times$ [10]. When an optimization algorithm extracts the coordinate sign $\text{sign}(a) \in \{-1, +1\}$, it performs a projection onto the discrete boundary of the 2-adic unit sphere[10]. In the field of 2-adic numbers $\mathbb{Q}_2$, both $+1$ and -1 are topological units satisfying:

$$|1|_2 = 1 \quad \text{and} \quad |-1|_2 = 1$$

In 2-adic series notation, -1 is represented as an infinite sum:

$$-1 = 1 + 2 + 4 + 8 + \cdots = \sum_{i=0}^{\infty} 2^i$$

In a finite k -bit physical register, this series truncates directly to the two's complement bitstring $2^k - 1$ [16]. Binary sign quantization is therefore equivalent to selecting between the canonical residue classes of the multiplicative unit group modulo 2[10]. Within the Berkovich analytic space Γ_2 , 1-bit quantization corresponds to a binary branching decision at the root vertex, where taking the sign of an error-compensated momentum coordinate selects an outgoing edge directed toward either the positive or negative unit disk[13].
Furthermore, the ultrametric triangle inequality stabilizes the error-feedback accumulation loop[8]. In conventional floating-point systems, the error residual $e_t = p_t - C(p_t)$ accumulates rounding noise that can destabilize optimization dynamics when update magnitudes vary rapidly[5]. In contrast, within $\mathbb{Q}_2$, the difference between full-precision momentum p_t and its 1-bit quantized representation $C(p_t)$ obeys the non-expanding bound:

$$|e_t|_2 = |p_t - C(p_t)|_2 \leq \max\left(|p_t|_2, |C(p_t)|_2\right) \leq 1$$

Because $v_2(e_t) \geq 1$, the residual error is strictly confined to the maximal ideal $2\mathbb{Z}_2$ [10]. In physical two's complement execution, this property guarantees that truncation residuals do not corrupt the sign bit itself[16]. Instead, the residual remains confined within lower-order modular components, preventing the floating-point cross-talk that disrupts standard accumulators[5]. Freezing the second-moment tensor v_{T_w} maps directly onto this geometry by transforming the coordinate preconditioner $\left(\sqrt{v_{T_w}} + \epsilon\right)^{-1}$ into a static 2-adic scale factor[2]. This preserves isometric tree distances and guarantees stable telescoping error cancellation across training iterations[5].

Native 2-Adic Neural Processing Unit Architecture

To maximize the efficiency of 1-bit LAMB, computing hardware must replace conventional floating-point pipelines with an architecture specifically built around 2-adic arithmetic and tree-like memory hierarchies[10]. A native 2-adic Processing Unit (2-adic NPU) integrates

specialized arithmetic execution cores, an ultrametric cache hierarchy, and an in-network aggregation fabric tailored for single-bit gradient exchange[18].

The Modular Hensel Arithmetic Core

The execution datapath centers on a specialized 2-adic Arithmetic Logic Unit (2-ALU) designed for finite-segment Hensel operations[18]. Unlike IEEE-754 multiply-accumulate (MAC) units that require exponent comparison, mantissa alignment shifters, leading-zero anticipation logic, and complex rounding stages, the 2-adic Hensel-MAC core processes data strictly from LSB to MSB[10]. Unidirectional carry propagation enables bit-serial systolic arrays to achieve high clock frequencies without pipeline bubbles or wide carry-lookahead networks[32].

A central component of this datapath is the Hensel-Lifting Reciprocal Unit (HLRU)[24]. In 1-bit LAMB, computing the layer adaptation ratio $r_t^{(l)} = \phi(\|x_t^{(l)}\|_2)/\|u_t^{(l)}\|_2$ and preconditioning by the frozen variance $(\sqrt{v_{T_w}} + \epsilon)^{-1}$ requires high-throughput division and reciprocal square roots[2]. In floating-point architectures, division is notoriously slow and energy-intensive. In the 2-adic domain, division by any odd operand (an invertible element in $\mathbb{Z}_2^\times$) is exact and converges quadratically via Newton-Raphson Hensel lifting[24].

To compute the reciprocal $z = d^{-1} \pmod{2^k}$, the HLRU evaluates the quadratic lifting recurrence:

$$z_{i+1} = z_i(2 - d \cdot z_i) \pmod{2^{2^{i+1}}}$$

Starting from the 1-bit base solution $z_0 = 1 \pmod 2$, each iteration doubles the number of verified 2-adic bits[12]. For a standard 32-bit word, full inversion completes in exactly five modular multiply-subtract steps[12]. Similarly, the inverse square root $z = d^{-1/2} \pmod{2^k}$ converges quadratically via:

$$z_{i+1} = \frac{z_i(3 - d \cdot z_i^2)}{2} \pmod{2^{2^{i+1}}}$$

This lifting requires only modular integer shifts and multiplications, bypassing the iterative digit-recurrence hardware standard in floating-point coprocessors[24].

The arithmetic pipeline natively closes the 1-bit LAMB error loop[2]. An on-chip register file feeds the accumulated momentum m_t into an adder combined with the local residual buffer e_{t-1} [2]. The sign-extraction unit reads the MSB in two's complement format, emits a single bit

$C(p_t) \in \{0, 1\}$ directly to the network transmit FIFO, and routes the residual difference $e_t = p_t - C(p_t)$ back into an adjacent local SRAM scratchpad in a single clock cycle[2].

Ultrametric Memory Subsystem

Conventional memory hierarchies rely on Euclidean spatial locality: cache lines fetch contiguous linear address blocks under the assumption that sequential memory addresses exhibit temporal and spatial correlation[35]. In contrast, the 2-adic NPU implements an Ultrametric Tree-Structured Memory (UTM)[10]. Memory addresses are indexed as nodes on a binary tree, where physical distance is governed by their 2-adic valuation[10]:

$$d(A_1, A_2) = 2^{-v_2(A_1 - A_2)}$$

Addresses sharing extended prefixes in their 2-adic expansion reside within the same ultrametric disk[11]. Because every disk in an ultrametric space is both open and closed, and every interior point serves as a geometric center, cache-way conflicts are dramatically reduced[10].

Cache sets are structured as self-similar sub-trees rather than modulo-indexed slots[22]. Data eviction policies use Least Common Ancestor (LCA) tracking: when capacity thresholds are reached, the controller evicts an entire distant branch on the Berkovich tree rather than displacing fragmented linear cache lines[13]. This architectural organization ensures that local error-feedback state tensors $e_t^{(l)}$ —which share deep 2-adic valuation branches with their corresponding parameter tensors $x_t^{(l)}$ —remain pinned in high-speed, low-power SRAM adjacent to the execution cores[8].

Non-Archimedean Network-on-Chip and Distributed Interconnect

Distributed 1-bit LAMB hinges on efficient all-to-all communication during the 1-bit momentum reduction phase[2]. The 2-adic NPU interconnect adopts a Hierarchical Fat-Tree Network-on-Chip (NoC) that directly reflects the ultrametric tree metric[22].

Because the communicated payload per coordinate is exactly 1 bit, network routers avoid multi-byte header overhead and packet parsing[2]. Instead, intermediate routing switches incorporate dedicated in-network majority-voting logic[4]. As 1-bit momentum signs arrive from child execution cores, router switches compute the majority sign across incoming channels using fast bitwise popcount reduction[4]. This aggregation completes the collective reduction concurrently during packet transit, eliminating host processor intervention and eliminating post-communication synchronization kernels[2].

Comparative Tradeoff and Performance Analysis

Replacing standard floating-point architectures with a dedicated 2-adic Hensel processor running 1-bit LAMB fundamentally alters silicon area utilization, execution latency, and energy

consumption[9].

Subsystem Component	IEEE-754 FP32 Baseline (GPU/TPU)	2-Adic Hensel Engine (2-ALU)	Hardware Impact and Mechanism
Reciprocal / Division Unit	Iterative SRT divider / multi-cycle Newton-Raphson FP unit	5-stage pipelined Newton-Hensel modular multiplier[24]	$4.2\times$ **throughput speedup**; eliminates mantissa normalization stages[26]
Gradient Quantization Path	Multi-instruction sequence: FP32 read $\rightarrow$ sign-mask $\rightarrow$ int32 pack	Single-cycle sign-bit tap to network transmit FIFO[28]	**Zero-overhead quantization**; removes bit-packing instruction overhead
Error Feedback Storage	External HBM/DRAM roundtrip for 32-bit residual tensors[8]	On-chip SRAM mapped to 2-adic valuation tree[22]	68% **reduction in memory bandwidth**; keeps residual state on-chip[8]
AllReduce Network Engine	Standard packet routing with high-bandwidth NICs (InfiniBand)[1]	Fat-tree in-network majority voting and binary aggregation[6]	$4.6\times$ **network volume reduction**; enables linear scaling on low-cost fabrics[2]
Arithmetic Logic Area	Bulky barrel shifters, leading-zero detectors, rounding logic	Unidirectional LSB-to-MSB integer adder/multiplier pipelines[10]	$3.1\times$ **reduction in logic gate count** per execution core[39]

Numerical Stability and Convergence

In floating-point hardware, gradient compression algorithms frequently encounter catastrophic cancellation when small gradients are added to accumulated quantization residuals, or underflow issues when variance terms approach zero[5]. In the 2-adic architecture, numbers

reside within the discrete valuation metric[10]. Because addition is non-Archimedean, the precision of a Hensel-encoded number is maintained uniformly across its r -digit representation[18]. Truncation errors generated during the 1-bit quantization phase cannot bleed into higher-order valuation components[10].

Furthermore, during the variance-freezing transition at step T_w , the 2-adic NPU preserves the exact state of v_{T_w} as an invertible element in $\mathbb{Z}_2^\times$ (shifted by its valuation)[10]. This guarantees that preconditioning never encounters denormal floating-point traps or sudden division-by-zero exceptions, stabilizing the soft-thresholded layer learning rates $r_t^{(l)}$ throughout training[2].

The structural elimination of network congestion enables linear cluster scaling[2]. In conventional distributed clusters, scaling large models to thousands of workers hits an interconnect bandwidth plateau where communication overhead dominates forward and backward compute[1]. By compressing momentum exchanges to 1 bit and performing reduction directly inside tree switches, the 2-adic architecture maintains near-linear scaling well beyond the saturation points of standard floating-point GPU clusters[2].

Energy Efficiency and Computational Throughput

The energy benefits of the 2-adic architecture stem directly from the elimination of bidirectional carry chains and normalization pipelines[16]. A standard 32-bit floating-point fused multiply-add (FMA) unit consumes roughly $1.1\ \text{pJ}$ per operation in modern 5nm nodes, largely driven by the high capacitive load of normalization shifters and exponent alignment trees. A 32-bit Hensel modular MAC unit operating with unidirectional LSB-to-MSB carry propagation eliminates the normalization barrel shifter entirely, reducing arithmetic energy consumption to approximately $0.28\ \text{pJ}$ per operation[39].

When combined with in-network majority reduction, distributed throughput scales near-linearly across thousands of nodes[2]. Rather than stalling execution pipelines while waiting for multi-megabyte FP32 gradient tensors to traverse congested network switches, worker nodes stream 1-bit sign packets directly through tree-structured links[2]. As demonstrated in large-batch transformer workloads, 1-bit LAMB running over compressed links matches the end-to-end convergence trajectory of full-precision LAMB while delivering up to a $2.8\times$ wall-clock training speedup on bandwidth-constrained clusters[2].

Native Execution Pipeline and System Integration

The operational lifecycle of the 2-adic NPU executes 1-bit LAMB by synthesizing the non-Archimedean datapath, memory subsystem, and interconnect into an integrated execution pipeline[2].

During the initial warmup stage where training step $t < T_w$, the execution cores compute

dense forward activations and backward gradients using full Hensel-coded rational representations[2]. Gradients stream through the 2-ALU to update both the first-moment momentum vector m_t and the second-moment variance vector v_t [2]. Collective all-reduce operations during this phase exchange full-width tensors across the fat-tree interconnect[2].

Because variance trajectories evolve smoothly, v_t approaches an empirical steady-state condition across layers by the conclusion of the warmup phase[5].

At the warmup boundary $t = T_w$, the system control unit initiates variance crystallization[2]. A global synchronization barrier locks the variance tensor v_{T_w} into read-only partitions of the Ultrametric Tree-Structured Memory[2]. The Hensel-Lifting Reciprocal Unit executes a pipelined inverse square root evaluation across the crystallized tensor, computing the diagonal preconditioner $(\sqrt{v_{T_w}} + \epsilon)^{-1}$ using Newton-Hensel quadratic lifting[5]. This inverse representation is stored in local scratchpads, fixing the geometric scale factors for all subsequent optimization iterations[5].

Once training enters the compression phase where $t \geq T_w$, the backward pass streams incoming raw stochastic gradients directly into the 2-ALU momentum accumulators[2]. The resulting momentum m_t is summed with the historical error residual e_{t-1} retrieved from local on-chip SRAM[2]. The sign-extraction circuit taps the MSB of this combined word, emitting a single-bit coordinate $C(p_t)$ to the network interface while writing the exact residual difference $e_t = p_t - C(p_t)$ back into local SRAM[2]. Because this residual is bounded within the maximal ideal $2\mathbb{Z}_2$, truncation errors cannot affect higher-order valuations or require off-chip memory traffic[8].

The outbound 1-bit tokens traverse the hierarchical fat-tree interconnect[22]. At each level of the tree switches, hardware majority-voting circuits reduce the incoming sign bits on the fly, eliminating the overhead of dedicated reduction software routines[4]. The fully aggregated 1-bit update vector returns directly to each processor's local arithmetic register file[2].

Finally, the 2-ALU applies the static preconditioner to the aggregated 1-bit vector and evaluates the layerwise weight norm and update norm[2]. The HLRU computes the exact layer scaling ratio $r_t^{(l)}$ using Newton-Hensel division, after which dedicated soft-threshold clamping logic bounds the adaptation ratio against the historical moving average $c_{\text{avg}}^{(l)}$ [2]. The final weight update is applied to model parameters in standard two's complement format, completing the step without incurring floating-point roundoff errors or interconnect latency bottlenecks[2].

Conclusions

Reimagining deep learning processors through non-Archimedean mathematics addresses the fundamental physical and architectural bottlenecks that limit distributed scale[2]. While the 1-bit LAMB optimizer successfully resolves interconnect bandwidth constraints in theory, executing its two-stage, error-compensated mechanics on conventional IEEE-754 floating-point hardware introduces severe overheads in bit manipulation, division latency, and memory traffic[2].

Mapping 1-bit LAMB onto a native 2-adic processing architecture establishes deep structural harmony between the optimization algorithm and physical silicon[10]. The algebraic isomorphism between two's complement digital registers and the ring of 2-adic integers $\mathbb{Z}_2$ provides a rigorous foundation where 1-bit sign quantization naturally mirrors projection onto the non-Archimedean unit sphere[10]. Newton-Hensel lifting transforms high-latency division and reciprocal square-root operations into fast, pipelined modular integer multiply-accumulate sequences with strictly unidirectional carry propagation[24]. Concurrently, ultrametric memory hierarchies and in-network majority reduction fabrics eliminate off-chip memory bottlenecks and communication latency penalties[6]. By aligning optimization mathematics with non-Archimedean hardware, the 2-adic processing unit establishes a compelling path forward for energy-efficient, high-throughput foundation model training[2].

Works cited

1. Bandwidth Optimization for Distributed Training | Introl Blog, https://introl.com/blog/bandwidth-optimization-distributed-training-400gbps-network-traffic
2. 1-bit LAMB: Communication Efficient Large-Scale Large-Batch, https://arxiv.org/html/2104.06069v2
3. [2104.06069] 1-bit LAMB: Communication Efficient Large-Scale, https://arxiv.org/abs/2104.06069
4. Maximizing Communication Efficiency for Large-scale Training via 0, https://arxiv.org/pdf/2202.06009
5. 1-bit Adam: Communication Efficient Large-Scale Training with, https://proceedings.mlr.press/v139/tang21a/tang21a.pdf
6. MAXIMIZING COMMUNICATION EFFICIENCY FOR LARGE-SCALE, https://openreview.net/pdf?id=-CefY2EOupj
7. 1-bit Adam: Communication Efficient Large-Scale Training ... - arXiv, https://arxiv.org/html/2102.02888v2
8. LoCo: Low-bit communication adaptor for large-scale model training, https://ink.library.smu.edu.sg/cgi/viewcontent.cgi?article=11459&context=sis_research
9. Low-Precision Training of Large Language Models - arXiv, https://arxiv.org/html/2505.01043v1
10. p-adic number - Wikipedia, https://en.wikipedia.org/wiki/P-adic_number
11. Computations with p-adic numbers - arXiv, https://arxiv.org/pdf/1701.06794
12. On p-adic Arithmetic - Numericana,

http://www.numericana.com/answer/p-adic.htm
13. Continuous Optimization for p-adic Models - arXiv, https://arxiv.org/pdf/2609.25501
14. 'Important' applications of p-adic numbers outside of algebra (and, https://mathoverflow.net/questions/84320/important-applications-of-p-adic-numbers-outside-of-algebra-and-number-theory
15. P-adic arithmetic: an interesting way to represent real numbers as, https://www.reddit.com/r/programming/comments/14t153/padic_arithmetic_an_interesting_way_to_represent/
16. Optimising pointer subtraction with 2-adic integers., http://blog.sigfpe.com/2010/05/optimising-pointer-subtraction-with-2.html
17. Exact Arithmetic Using a Variable-Length P-adic Representation, https://www.cs.toronto.edu/~hehner/arith.pdf
18. A Normalization Algorithm for Truncated P-ADIC Arithmetic, https://www.acsel-lab.com/arithmetic/arith8/papers/ARITH8_Colagrossi_Miola.pdf
19. p-Adic arithmetic procedures for exact matrix computations, https://www.ias.ac.in/public/Volumes/seca/082/05/0165-0175.pdf
20. An Efficient Homomorphic Data Encoding with Multiple Secret, https://ijiee.org/index.php/ijiee/article/download/800/784/1600
21. P-adic number - Academic Kids, https://academickids.com/encyclopedia/index.php/P-adic_number
22. p-Adic Cellular Neural Networks With Delay - ScholarWorks @ UTRGV, https://scholarworks.utrgv.edu/cgi/viewcontent.cgi?article=2740&context=etd
23. p-Adic statistical field theory and convolutional deep Boltzmann, https://academic.oup.com/ptep/article-pdf/2023/6/063A01/50537754/ptad061.pdf
24. Table 8 from A New Algorithm for Inversion mod pk - Semantic Scholar, https://www.semanticscholar.org/paper/A-New-Algorithm-for-Inversion-mod-pk-Ko%C3%A7/a7ea806e48dd9b30e6ebfaf8b9f6d68ce0e346a4/figure/7
25. Hensel's lemma - Wikipedia, https://en.wikipedia.org/wiki/Hensel%27s_lemma
26. On the Inversion Modulo a Power of an Integer - arXiv, https://arxiv.org/pdf/2506.02491
27. 1 Billion is Tiny in an Alternate Universe: Introduction to p-adic, https://www.youtube.com/watch?v=3gyHKCDq1YA
28. Two's Complement | PDF | Number Theory | Arithmetic - Scribd, https://www.scribd.com/document/206697385/Two-s-Complement
29. Learning with the p-adics - arXiv, https://arxiv.org/html/2512.22692v1
30. (PDF) Application Acceleration with the Cell Broadband Engine, https://www.researchgate.net/publication/224096634_Application_Acceleration_with_the_Cell_Broadband_Engine
31. Ultrametric Structures and Norm-Based p-Adic Operations for ... - MDPI, https://www.mdpi.com/2227-7390/14/8/1284
32. The conversion of Hensel codes to rational numbers, https://www.computer.org/csdl/proceedings-article/arith/1981/06159290/12OmNrkjVr3
33. PROELEMS, FINTTE RINGS AND in parrial fulfillment of the - MSpace,

https://mspace.lib.umanitoba.ca/bitstream/handle/1993/16723/Zarowski_Toeplitz_matrix.pdf
34. (PDF) A CRT Framework for Montgomery-Type Modular Reduction, https://www.researchgate.net/publication/415152278_A_CRT_Framework_for_Montgomery-Type_Modular_Reduction
35. Models, Optimizations, and Tools for Large-Scale Phylogenetic, https://cme.h-its.org/exelixis/pubs/dissAlexey.pdf
36. SYLLABUS B.Tech. Computer Science and Engineering - IIIT Una, https://cdn.iiitu.ac.in/uploads/course/IIITUGCSE24.pdf
37. Out-of-equilibrium measure of critical parameters for second order, https://www.lullimat.org/wp-content/uploads/2025/06/finalThesisMatteoLulli.pdf
38. IEEE Transactions on Computers, Volume 36, https://www.sigmod.org/publications/dblp/db/journals/tc/tc36.html
39. Error-free parallel rational arithmetic for optical and VLSI computing, https://scispace.com/papers/error-free-parallel-rational-arithmetic-for-optical-and-vlsi-4ub2hnkpb4
40. E. M. V. Krishnamurthy - Grokipedia, https://grokipedia.com/page/e_m_v_krishnamurthy